

Examen Módulo 2

24/04/2023

Nombre: _____

Instrucciones

El presente examen tiene tres partes. Cada parte indica su puntaje y el nombre del archivo que se debe entregar. Tendrás un tiempo de 110 minutos para resolver el examen.

Debes entregar a través de Canvas un archivo para cada parte con el nombre que se especifica en el enunciado. **NO SE ACEPTARÁN ENTREGAS POR CORREO ELECTRÓNICO. POR LO TANTO, DEBES ANTICIPARTE Y ENTREGAR POR CANVAS 5 A 10 MINUTOS ANTES DEL CIERRE DEL BUZÓN. PUEDES ENTREGAR LAS VECES QUE ESTIMES CONVENIENTE, HASTA EL MOMENTO DEL CIERRE.**

Evaluación

Cada ítem en este examen es evaluado con una escala de 1-5 (1: no presenta solución, 2: presenta esbozo de solución incompleto o con errores graves, 3: solución incompleta o imprecisa, 4: error(es) mínimo(s), 5: excelente) y luego un ponderador (1:0, 2:0,25, 3:0,5, 4:0,75, 5:1,0) es aplicado al puntaje del ítem. El puntaje otorgado a la solución se calcula como la suma de los puntajes parciales obtenidos, aplicándose los ponderadores.

Parte 1 (2 puntos)

1. [.5] Crea un programa en Python (**p1-1.py**) que pida al usuario un valor entero n y calcule y despliegue el resultado de la serie alternante de los primeros n impares:

$$1 - 3 + 5 - 7 + 9 - \dots$$

Puedes suponer que en el programa siempre se ingresará $n \geq 1$.

2. [.5] Gottfried Leibniz y otros matemáticos antes que él, determinaron que la constante π es aproximable mediante la siguiente serie:

$$1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \dots = \frac{\pi}{4}$$

Crea un programa (archivo **p1-2.py**) que calcule y despliegue π considerando la serie anterior con n términos. Para esto, el programa debe pedir el valor n al usuario.

3. [1.0] Crea una copia del programa anterior (archivo **p1-3.py**) que pida un valor n para aproximación de π usando la serie anterior, valores reales a y b , extremos del intervalo $[a, b]$ con (se puede suponer que siempre se ingresará $b > a$), y un valor real positivo w , tal que $w < 1$ (se puede suponer que siempre se ingresa válido). Tu programa debe generar todos los valores de $\sin(a + kw\pi)$, en donde k es entero que toma valores desde 0 hasta el mayor entero posible tal que $a + kw\pi \leq b$. Tu programa debe imprimir en líneas sucesivas cada valor $a + kw\pi$ generado en el intervalo $[a, b]$, junto con el valor

de $\sin(a + kw\pi)$, separando estos valores con una coma.

Ejemplo de ejecución:

```
n> 1000
a> 0
b> 20
w> 0.5
0.0,0.0
1.570296326919897,0.9999998750000652
3.140592653839794,0.00099999958333257
4.710888980759691,-0.9999988750007734
6.281185307679588,-0.0019999981666661403
7.851481634599486,0.9999968750031901
9.421777961519382,0.0029999947500031544
10.99207428843928,-0.9999938750093151
12.562370615359177,-0.00399998833346281
14.132666942279073,0.9999898750221484
15.702963269198971,0.004999977916703407
17.27325959611887,-0.9999848750456901
18.843555923038764,-0.0059999625000873085
```

Nota importante: Para usar la función $\sin(x)$ en Python debes incorporar la siguiente línea al inicio de tu programa:

```
from math import sin
```

Esta función opera con ángulos en radianes.

Solución

Listing 1: Solución del problema 1

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  n = int(input("n> "))
5
6  signo = 1
7
8  i = 1
9  suma = 0
10 while i <= n:
11     suma += signo * (2*i-1)
12     signo *= -1
13     i += 1
14
15 print(suma)
```

Listing 2: Solución del problema 2

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  n = int(input("n> "))
5
6  signo = 1
7
8  i = 1
9  suma = 0
10 while i <= n:
11     suma += signo/(2*i-1)
```

```
12     signo *= -1
13     i += 1
14     pi = suma * 4
15
16     print(pi)
```

Listing 3: Solución del problema 3

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  from math import sin
4
5  n = int(input("n> "))
6
7  signo = 1
8
9  i = 1
10 suma = 0
11 while i <= n:
12     suma += signo/(2*i-1)
13     signo *= -1
14     i += 1
15 pi = suma * 4
16
17 a = float(input("a> "))
18 b = float(input("b> "))
19 w = float(input("w> "))
20
21 k = 0
22
23 while a+k*w*pi <= b:
24     print(f"{a+k*w*pi},{sin(a+k*w*pi)}")
25     k += 1
```

Parte 2 (2.5 puntos)

En esta segunda parte, simularás el movimiento circular uniforme de dos partículas A y B centradas en el origen, y con trayectorias de igual radio r . Las partículas tienen velocidades angulares distintas ω_A , ω_B , respectivamente, en rad/s.

Para lo anterior, considera que las posiciones de las partículas en el plano cartesiano x - y son dadas por las siguientes ecuaciones:

$$x_A(t) = r \cos(\omega_A t)$$

$$y_A(t) = r \sin(\omega_A t)$$

De las ecuaciones se entiende que en el tiempo t las coordenadas (x_A, y_A) de la partícula A están dadas por las funciones de coseno y seno, y la velocidad angular respectiva.

La posición inicial de ambas partículas es la misma, en $(r, 0)$.

En el caso de la partícula B , análogamente:

$$x_B(t) = r \cos(\omega_B t)$$

$$y_B(t) = r \sin(\omega_B t)$$

En Python para usar la velocidad angular en rad/s con las funciones trigonométricas, puedes usar el ángulo directamente en radianes. Para incluir las funciones trigonométricas necesarias debes agregar al inicio de tu programa lo siguiente:

```
from math import sin, cos
```

Requisitos:

- Crea en tu programa una lista llamada **parametros**, en donde deberás ingresar manualmente las constantes requeridas por las ecuaciones de movimiento, en el siguiente orden: r , ω_A y ω_B . Para probar tu programa, puedes ingresar manualmente a la lista los valores 4 (para r), 1.57 (para velocidad angular $\omega_A = \frac{\pi}{2}$), y 0.785 ($\omega_B = \frac{\pi}{4}$), considerando $\pi = 3,1416$.
- El programa al iniciarse debe pedir al usuario el tiempo total t a simular ($t \geq 1$, t entero), en segundos. Si el usuario ingresa un valor menor que 1, se le debe mostrar error y volver a pedir.
- Además, se debe pedir al usuario un valor entero f ($f \geq 1$, entero)¹. También se debe volver a pedir el valor f al usuario si se ingresa incorrectamente.
- La simulación tiene que desarrollarse iterativamente (con un ciclo), y en cada iteración de la simulación el tiempo simulado debe avanzar en $1/f$. La simulación debe iterar hasta que el tiempo transcurrido supere a t .
- En cada iteración, la simulación debe generar las posiciones de A y B en el tiempo actual simulado, usando las ecuaciones de movimiento mostradas arriba.
- El programa debe generar cuatro listas llamadas XA , XB , YA , YB , que contendrán todos los puntos cartesianos generados en las trayectorias de A y B en la simulación, con las ecuaciones de movimiento.
- Cada punto de la partícula A en la simulación debe quedar en las listas XA e YA . Para la partícula B , se deben usar las listas XB e YB . Al término de la simulación, todas las listas deben tener el mismo tamaño, considerando que se genera igual cantidad de puntos para cada partícula en la simulación.
- Para terminar, el programa debe mostrar los puntos de las trayectorias de las partículas A y B como coordenadas cartesianas, incluyendo el tiempo de los puntos. Ejemplo:

```

t> 10
f> 4
T:0.00 A:(4.00,0.00) B:(4.00,0.00)
T:0.25 A:(3.70,1.53) B:(3.92,0.78)
T:0.50 A:(2.83,2.83) B:(3.70,1.53)
T:0.75 A:(1.53,3.69) B:(3.33,2.22)
T:1.00 A:(0.00,4.00) B:(2.83,2.83)
T:1.25 A:(-1.53,3.70) B:(2.22,3.32)
...
```

En el ejemplo, se ve que las coordenadas aparecen con dos dígitos decimales. Una manera de lograr esto es usar f-strings en Python. Ejemplo: `print(f'val:.2f')` despliega la variable `val` con 2 posiciones decimales.

Nota: Es importante considerar que la simulación debe realizarse en forma instantánea, esto es, sin simular el tiempo real transcurrido. Así, en la práctica tomará mínimo tiempo al computador simular, por ejemplo, 60 segundos.

¹Esto corresponde a la frecuencia con que se generarán los puntos de las trayectorias de A y B . El valor de f está en Hz.

Desglose de puntaje:

- [.5] El programa pide los parámetros t y f al usuario, y valida cada uno de ellos. Es decir muestra un error cuando éstos son mal ingresados, y pide al usuario volverlos a ingresar, repitiendo este proceso hasta que ambos parámetros estén correctos.
- [.5] El programa implementa una iteración para simular el transcurso del tiempo en la simulación y la generación de puntos de trayectoria de partículas A y B con frecuencia f .
- [.5] El programa evalúa correctamente las ecuaciones de movimiento utilizando constantes provenientes de la lista `parametros`, y el tiempo simulado.
- [.5] El programa inicializa y genera correctamente los valores en las listas XA , YA , XB , YB .
- [.5] El programa despliega correctamente las coordenadas de cada partícula al finalizar la simulación, incluyendo la posición de ambas partículas y el tiempo.

Solución

Listing 4: Solución del problema

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  from math import sin, cos
4
5  XA = []
6  YA = []
7  XB = []
8  YB = []
9
10 parametros = [4, 1.57, .785]
11
12 t = int(input("t> "))
13 while (t<1):
14     print("ingrese t >= 1")
15     t = int(input("t> "))
16
17 f = int(input("f> "))
18 while (f<1):
19     print("ingrese f >= 1")
20     f = int(input("f> "))
21
22 tf = 1/f
23 tsim = 0
24
25 while tsim <= t:
26     XA.append(parametros[0]*cos(parametros[1]*tsim))
27     YA.append(parametros[0]*sin(parametros[1]*tsim))
28     XB.append(parametros[0]*cos(parametros[2]*tsim))
29     YB.append(parametros[0]*sin(parametros[2]*tsim))
30     tsim += tf
31
32 i = 0
33 tprint = 0
34 while i < len(XA):
35     print(f"T:{tprint:.2f} A:({XA[i]:.2f},{YA[i]:.2f}) B:({XB[i]:.2f},{YB[i]:.2f})")
36     tprint += 1/f
37     i+=1
```

Parte 3 (1.5 puntos)

Esta última parte del examen requiere haber completado la Parte 2. Para continuar, crea una copia del programa de la parte anterior, con nombre `p3.py`.

Extiende el programa anterior para que determine y despliegue al finalizar la ejecución:

- [1.0] La mayor distancia entre las partículas A y B y el instante de tiempo en que ésta ocurre. Usa para esto la distancia euclidiana d entre dos puntos (x_0, y_0) , (x_1, y_1) , definida como:

$$d = \sqrt{(x_1 - x_0)^2 + (y_1 - y_0)^2}$$

Para esto, sólo debes considerar los puntos de trayectoria generados en la simulación, y que se encuentran en las listas XA , XB , YA , e YB .

- [.5] La distancia euclidiana promedio entre las partículas A y B durante la simulación.

Ejemplo de ejecución:

```
t> 1
f> 8
T:0.00 A:(4.00,0.00) B:(4.00,0.00)
T:0.12 A:(3.92,0.78) B:(3.98,0.39)
T:0.25 A:(3.70,1.53) B:(3.92,0.78)
T:0.38 A:(3.33,2.22) B:(3.83,1.16)
T:0.50 A:(2.83,2.83) B:(3.70,1.53)
T:0.62 A:(2.22,3.32) B:(3.53,1.88)
T:0.75 A:(1.53,3.69) B:(3.33,2.22)
T:0.88 A:(0.78,3.92) B:(3.09,2.54)
T:1.00 A:(0.00,4.00) B:(2.83,2.83)
Distancia máxima: 3.06 en tiempo 1.00
```

Solución

Listing 5: Solución del problema

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  from math import sin, cos
4
5  XA = []
6  YA = []
7  XB = []
8  YB = []
9
10 parametros = [4, 1.57, .785]
11
12 t = int(input("t> "))
13 while (t<1):
14     print("ingrese t >= 1")
15     t = int(input("t> "))
16
17 f = int(input("f> "))
18 while (f<1):
19     print("ingrese f >= 1")
20     f = int(input("f> "))
21
22 tf = 1/f
23 tsim = 0
24
25 tdmx = 0
26 dmx = 0
27 dsum = 0
28
29 while tsim <= t:
```

```
30     xa = parametros[0]*cos(parametros[1]*tsim)
31     ya = parametros[0]*sin(parametros[1]*tsim)
32     xb = parametros[0]*cos(parametros[2]*tsim)
33     yb = parametros[0]*sin(parametros[2]*tsim)
34
35     d = ((xa-xb)**2 + (ya-yb)**2) ** .5
36     dsum += d
37
38     if d > dmax:
39         dmax = d
40         tdmax = tsim
41
42     XA.append(xa)
43     YA.append(ya)
44     XB.append(xb)
45     YB.append(yb)
46
47     tsim += tf
48
49     davg = dsum/len(XA)
50
51     i = 0
52     tprint = 0
53     while i < len(XA):
54         print(f"T:{tprint:.2f} A:({XA[i]:.2f},{YA[i]:.2f}) B:({XB[i]:.2f},{YB[i]:.2f})")
55         tprint += 1/f
56         i+=1
57
58     print(f"Distancia máxima: {dmax:.2f} en tiempo {tdmax:.2f}")
```
